

TLS Certificate Trust Setup

Trusting the E-PAY self-signed certificate on Windows

The Problem

The Flask development server uses a **self-signed TLS certificate**. Because it was not issued by a recognised Certificate Authority, browsers block it by default. You will see a warning like *Your connection is not private* before you can reach the app at all.

The fix is simple: tell Windows to trust the certificate by adding it to your Trusted Root store. Once done, browsers accept it without further complaints.

Step 1 - Locate the Certificate

The certificate ships with the project and is already in place at:

```
E_PAY/nginx/ssl/selfsigned.crt
```

```
E_PAY/nginx/ssl/selfsigned.key
```

Step 2 - Add to Your User Trusted Root Store

Open PowerShell and run the command below. It adds the certificate to your **current user** store, so it only affects you and does not require administrator rights:

```
certutil -addstore -user Root "E_PAY\nnginx\ssl\selfsigned.crt"
```

Part	What it does
certutil	Windows built-in certificate management utility
-addstore	Adds a certificate into a named store
-user	Targets the current user store, not the whole machine
Root	The Trusted Root Certification Authorities store
the .crt file	The self-signed certificate you want Windows to trust

In plain terms, this command tells Windows to personally vouch for the certificate and treat it as if it came from a trusted CA.

Step 3 - (If Needed) Add to the Local Machine Store

Chrome and Edge sometimes also check the **Local Machine** certificate store. If the browser still warns you after Step 2, add the cert machine-wide. This requires administrator privileges.

Option A - From a regular PowerShell prompt (Windows will prompt for your admin password):

```
Start-Process -Verb RunAs -FilePath "certutil" -ArgumentList "-addstore","Root","E_PAY\nginx\ssl\selfsigned.crt"
```

Option B - From an already-elevated admin PowerShell prompt:

```
certutil -addstore Root "E_PAY\nginx\ssl\selfsigned.crt"
```

Step 4 - Restart Your Browser

Browsers load certificate trust settings at startup and do not pick up changes mid-session. **Close your browser completely and reopen it** for the new trusted certificate to take effect.

Verify It Worked

With the server running, hit the health endpoint from PowerShell:

```
curl.exe https://localhost:5001/health
```

State	What you see
Before trusting	curl: (60) SEC_E_UNTRUSTED_ROOT -or- browser shows 'Your connection is not private'
After trusting	{"message": "E-Banking API is running", "status": "ok"} with no errors

Why Browsers Block Self-Signed Certificates

Browsers only trust certificates issued by a public Certificate Authority such as Let's Encrypt or DigiCert. A self-signed certificate has no such endorsement; it is simply the server asserting its own identity with no outside verification.

When you add the certificate to the Windows Trusted Root store, you become the authority. You are personally telling the operating system that you have reviewed and trust this specific certificate. The browser then takes your word for it.

How to Undo (Remove the Certificate)

If you ever want to remove the trust, follow these steps:

1. Find the certificate thumbprint:

```
certutil -store -user Root localhost
```

2. Remove from the Current User store:

```
certutil -delstore -user Root &lt;thumbprint&gt;
```

3. Remove from the Local Machine store (requires admin):

```
certutil -delstore Root &lt;thumbprint&gt;
```

Alternative - Skip TLS Entirely

If you would rather not deal with certificates during development, you can disable TLS altogether. The server will run over plain HTTP:

```
$env:ENABLE_TLS="0"
```

```
python e_banking\app.py
```

Note: The server will start on `http://localhost:5001` with no encryption. This is fine for local development, but never disable TLS in a production or shared environment.